

```

Speech2Text agent
# You may need to add your working directory to the Python path. To do so,
uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agent-framework") # Replace with
your directory path

# Besser Agentic Framework Hugging Face Speech-to-text example agent

# imports
import logging
import base64

from baf.core.agent import Agent
from baf.core.session import Session
from baf.exceptions.logger import logger

from baf.nlp.llm.llm_openai_api import LLMOpenAI
from baf.nlp.speech2text.hf_speech2text import HFSpeech2Text

from baf.core.file import File
from baf.library.transition.events.base_events import ReceiveFileEvent

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('Huggingface Speech-to-Text Agent')

# Load agent properties stored in a dedicated file
agent.load_properties('config.yaml')

# example models
# 'Lemsasabi/wav2vec2-large-xlsr-53-842h-luxembourgish-14h-with-lm'
# 'openai/whisper-tiny'
# 'openai/whisper-large-v3'

# Define the platform your agent will use
websocket_platform = agent.use_websocket_platform(use_ui=True)

# Define STT Models
stt = HFSpeech2Text(agent=agent, model_name="openai/whisper-tiny")

# Create the LLM
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4o-mini',
    parameters={},
    num_previous_messages=100,
)

# States

```

```

initial_state = agent.new_state('initial_state', initial=True)
awaiting_state = agent.new_state('awaiting_state') # for awaiting user
input
stt_state = agent.new_state('stt_state') # for messages and speech
stt_file_state = agent.new_state('stt_file_state') # for audio files
uploaded through the UI

# STATES BODIES' DEFINITION + TRANSITIONS

def initial_body(session: Session):
    answer = gpt.predict(
        f"You are a helpful assistant. Start the conversation with a short
        (2-15 words) greetings message. Make it original.")
    session.reply(answer)

initial_state.set_body(initial_body)
initial_state.go_to(awaiting_state)

def awaiting_body(session: Session):
    pass

awaiting_state.set_body(awaiting_body)
awaiting_state.when_file_received(allowed_types=("audio/wav", "audio/mpeg",
"audio/mp4")).go_to(stt_file_state) # Only Allow Wav, MP3, MP4 files
awaiting_state.when_no_intent_matched().go_to(stt_state)

def stt_body(session: Session):
    session.reply("User: " + session.event.message)
    answer = gpt.chat(session)
    session.reply(answer)

stt_state.set_body(stt_body)
stt_state.go_to(awaiting_state)

# Execute when a file is received
def stt_file_body(session: Session):
    # get user language
    lang = session.get("user_language", "en")
    # access STT system based on language mapping
    s2t = session._agent._nlp_engine._language_to_speech2text_module[lang]
    event: ReceiveFileEvent = session.event
    file: File = event.file

    # convert file to byte representation
    base64_content = file._base64
    # Decode the base64 string into bytes
    file_bytes = base64.b64decode(base64_content)
    # add to logger

```

```
    logger.info(f"Successfully decoded {len(file_bytes)} bytes for  
Speech2Text.")
```

```
    # call HF Speech2Text and get transcription  
    text = s2t.speech2text(file_bytes)  
    session.reply("User: " + text)  
    answer = gpt.predict(text)  
    session.reply(answer)
```

```
stt_file_state.set_body(stt_file_body)  
stt_file_state.go_to(awaiting_state)
```

```
if __name__ == '__main__':  
    agent.run()
```